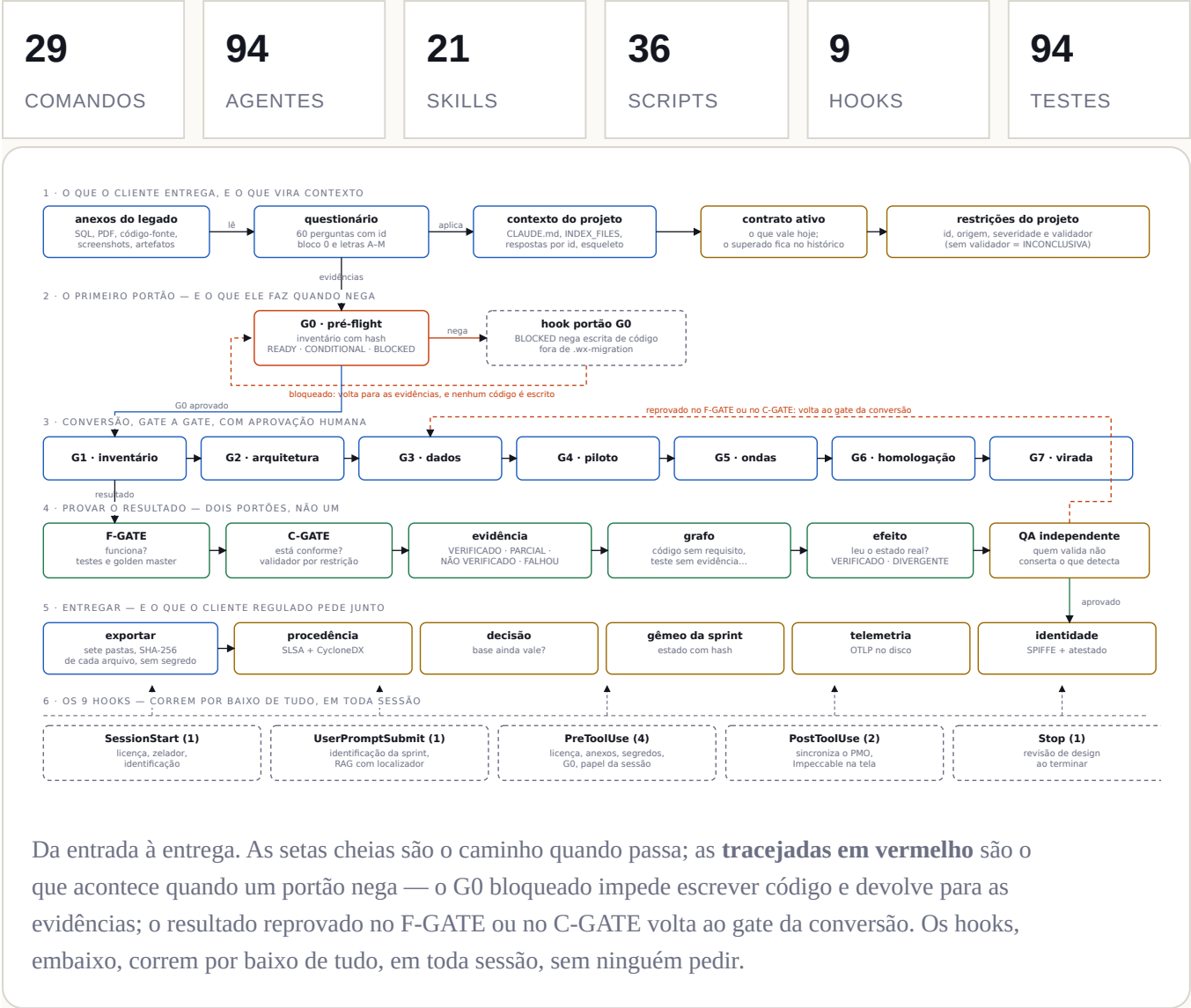


# Fluxo atual do WX Claude Code

Versão 3.35.0, medida em 2026-09-06. O desenho mostra o mecanismo, não a lista de peças: onde cada portão nega passagem e para onde a coisa volta quando ele nega. Portão desenhado como caixa no meio da linha não explica nada; a seta de volta explica.



Da entrada à entrega. As setas cheias são o caminho quando passa; as **tracejadas em vermelho** são o que acontece quando um portão nega — o G0 bloqueado impede escrever código e devolve para as evidências; o resultado reprovado no F-GATE ou no C-GATE volta ao gate da conversão. Os hooks, embaixo, correm por baixo de tudo, em toda sessão, sem ninguém pedir.

## As guardas, que rodam sozinhas

Regra que depende de alguém lembrar não é regra. Estas rodam nos eventos do Claude Code, e cada negativa entra no registro de operações. A do **papel da sessão** é a mais nova e segue a regra da casa: sem papel declarado, nada muda para quem já usava o plugin.

| GUARDA    | EVENTO     | O QUE FAZ VALER                                                         |
|-----------|------------|-------------------------------------------------------------------------|
| portão G0 | PreToolUse | nenhuma escrita fora de .wx-migration antes do G0 passar; falha fechado |
|           |            |                                                                         |

| GUARDA                      | EVENTO                                                    | O QUE FAZ VALER                                                                               |
|-----------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| papel da sessão             | <a href="#">PreToolUse</a>                                | com WX_PAPEL=qa, quem valida não escreve o produto que valida; sem papel declarado, nada muda |
| guarda de anexos e segredos | <a href="#">PreToolUse</a>                                | inputs/ e artefatos/ somente leitura; token em arquivo ou comando é recusado                  |
| licença                     | <a href="#">PreToolUse</a> · <a href="#">SessionStart</a> | serial confere a máquina antes de rodar script do plugin                                      |
| sincronizar PMO             | <a href="#">PostToolUse</a>                               | Kanban acompanha a matriz sem ninguém digitar                                                 |
| identificação               | <a href="#">UserPromptSubmit</a>                          | BlocoNNNN-SPNNNNN-Título · data em toda resposta                                              |
| RAG                         | <a href="#">UserPromptSubmit</a>                          | injeta o trecho mais próximo com arquivo#linha, e o tema do Help                              |
| zelador                     | <a href="#">SessionStart</a>                              | limpa temporário uma vez por dia e mede o que liberou                                         |
| impeccable                  | <a href="#">PostToolUse</a> · <a href="#">Stop</a>        | revisão de design da tela que acabou de mudar                                                 |

## O que roda antes de cada commit

Três provas, em níveis que não se substituem: a bateria pega a peça quebrada, o fluxo pega a peça certa ligada errada, e os cenários pegam o caminho que ninguém imaginou.

| PROVA                              | TAMANHO     | O QUE PEGA                                                                                                         |
|------------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------|
| tests/testes.py                    | 94 testes   | cada peça isolada: validação, hooks, licença, PMO, roteador, evidência, restrições, grafo, procedência             |
| tests/fluxo.py                     | 2 passos    | o caminho inteiro num projeto novo: questionário → contexto → G0 → artefato → PDF → PMO → RAG → entrega → registro |
| tests/cenarios.py                  | 18 cenários | os outros caminhos: sem licença, PDF que é foto, legado PHP, F-GATE verde com C-GATE reprovado, entrega auditável  |
| validate_plugin_bundle.py --strict | 1 comando   | arquivos obrigatórios, manifesto, e roda a bateria por dentro                                                      |
| claude plugin validate             | 1 comando   | o manifesto pelos olhos do próprio Claude Code                                                                     |

## As oito etapas, do install à entrega

O desenho mostra o mecanismo; a lista mostra a ordem em que um projeto passa por ele. Os números saem do repositório, não desta prosa.

## 1 Instalar e ativar `claude plugin install`

**licença** serial confere a máquina; sem ele, os scripts do plugin recusam

**comandos** o índice: 29 comandos e os 60 ids das perguntas

## 2 Perguntar `/wx-claude-code:questionario`

**bloco 0** empresa, diretores, logotipos, prazo, orçamento, riscos, GitHub, aprovador (0.1–0.16)

**A–E** evidências: SQL da análise e os PDFs — ou o código-fonte, quando o legado não é WX

**F** tela modelo e estilo (F0–F13)

**G** corpus do Help WLanguage (12k)

**H · I** destino: qualquer linguagem; perfil «outra» é aceito

**J** economia de tokens e modelo local

**K** ambiente (K0–K7) e backup e replicação (K8)

**L** kickoff, hooks, MCP, implantação, esqueleto de ERP (L1–L6)

**M** artefatos do cliente, um por vez, com onde usar

## 3 Gerar o contexto `aplicar_questionario.py`

**CLAUDE.md · INDEX\_FILES.md** o que a primeira sessão lê antes de qualquer comando

**respostas\_questionario.md** as respostas com índice por id, para os agentes

**esqueleto** AGENTS, CONTEXT, ADRs, domínios, database, src, tests, workflows

**102 arquivos** nada é sobrescrito ao reaplicar

## 4 Provar a entrada `/wx-claude-code:preflight – G0`

**inventário** cada evidência com hash e classificação

**legado sem WX** o código-fonte é a evidência central; PDF não é exigido de quem nunca o teve

**relatório** o hook portão G0 falha fechado se não conseguir lê-lo

## 5 Converter `/wx-claude-code:converter – G1 a G7`

|                      |                                                        |                       |                                                   |
|----------------------|--------------------------------------------------------|-----------------------|---------------------------------------------------|
| <b>G1 inventário</b> | BR-, QRY-, UI-, RPT-, INT-, DB- com origem localizável | <b>G2 arquitetura</b> | o que cada peça vira; a estratégia escolhida em H |
| <b>G3 dados</b>      | esquema migrado, chaves e integridade                  | <b>G4 piloto</b>      | uma vertical inteira, com golden master           |
| <b>G5 ondas</b>      | módulo a módulo, telas pelo Impeccable                 | <b>G6 homologação</b> | paralelo com o legado                             |
| <b>G7 virada</b>     | só com backup do legado restaurado, não copiado        |                       |                                                   |

## 6 Provar o resultado `constraints · evidencia · grafo · efeito`

|                        |                                                                               |                  |                                                            |
|------------------------|-------------------------------------------------------------------------------|------------------|------------------------------------------------------------|
| <b>F-GATE e C-GATE</b> | funciona? e está conforme? — a Sprint precisa dos dois                        | <b>evidência</b> | quatro estados, e o que ela NÃO prova é campo obrigatório  |
| <b>grafo</b>           | código sem requisito, requisito sem teste, teste sem evidência, prova vencida | <b>efeito</b>    | lê o estado real; inconclusivo tem código de saída próprio |

## 7 Governar `/wx-claude-code:pmo · contrato`

|                        |                                              |                       |                                                                                            |
|------------------------|----------------------------------------------|-----------------------|--------------------------------------------------------------------------------------------|
| <b>sprints</b>         | BlocoNNNN-SPNNNNN, cada uma em .md zipado    | <b>contrato ativo</b> | o que vale hoje; o superado fica no histórico com o motivo                                 |
| <b>papel da sessão</b> | quem valida não escreve o produto que valida | <b>equipe A–J</b>     | zelador, pesquisador, documentador, qualidade, tarefas, GP, testes, status, base, tradutor |

## 8 Entregar e auditar `/wx-claude-code:exportar · procedencia`

|                        |                                                                       |                    |                                                                       |
|------------------------|-----------------------------------------------------------------------|--------------------|-----------------------------------------------------------------------|
| <b>sete pastas</b>     | manifesto com SHA-256 de cada arquivo, sem segredo                    | <b>procedência</b> | SLSA e CycloneDX, com o nível de SLSA marcado INDISPONÍVEL e o porquê |
| <b>gêmeo e decisão</b> | a sprint fotografada e a base de cada decisão, para reconferir depois | <b>identidade</b>  | SPIFFE assinado e o atestado — que declara não ser attestation        |

## Onde as coisas caem

`.wx-migration/` é a governança — respostas, matriz, contrato ativo, restrições, evidências, decisões capturadas, gêmeos de sprint, procedência, telemetria, PMO, prompts e logs. `inputs/` é a evidência do legado, **somente leitura**; `artefatos/` é o que o cliente mandou por fora, também somente leitura e com hash. A raiz recebe o que a primeira sessão lê ( `CLAUDE.md` , `INDEX_FILES.md` ) e o esqueleto do projeto. O organograma de arquivos detalha os 102 arquivos que o questionário pode gerar.

Esta página é gerada por `docs/dossie/gerar-fluxo.py` e atualizada junto das outras por `docs/dossie/atualizar-paginas.py`. Ela era a última mantida à mão — e ficou seis lançamentos atrás, dizendo «19 comandos» quando já havia 29. Número visível sai de gerador, ou envelhece calado.